

LLMOps Platform

System Architecture

EKS-based internal AI gateway with multi-provider routing, cost governance, and full observability

Version 1.0 Date: 2026-06-12 Author: sirfenrir Audience: Mentor / Evaluator

Contents

1. Platform Overview
2. AWS Infrastructure Layer
3. GitOps & Deployment Layer
4. LLM Gateway — LiteLLM
5. Application Services
6. Observability Stack
7. Security & Governance
8. Traffic Simulation & Chaos Testing
9. Architecture Tradeoffs & Decisions

1. Platform Overview

This platform provides a secure, observable, and cost-governed internal LLM service for a 290-person engineering organisation. Employees interact through a web chat UI; all LLM traffic flows through a centralised proxy gateway before reaching any external model provider.

Goal	Implementation
Internal-only access	All services exposed only via internal AWS ALB — no public endpoints
Provider abstraction	Model aliases hide provider details; users never see raw model names
Cost governance	Per-team monthly budgets and rate limits enforced at the gateway
Full observability	LLM traces (Langfuse), infra metrics (Prometheus/Grafana), logs (Loki)
GitOps operations	All configuration in Git; ArgoCD continuously reconciles cluster state

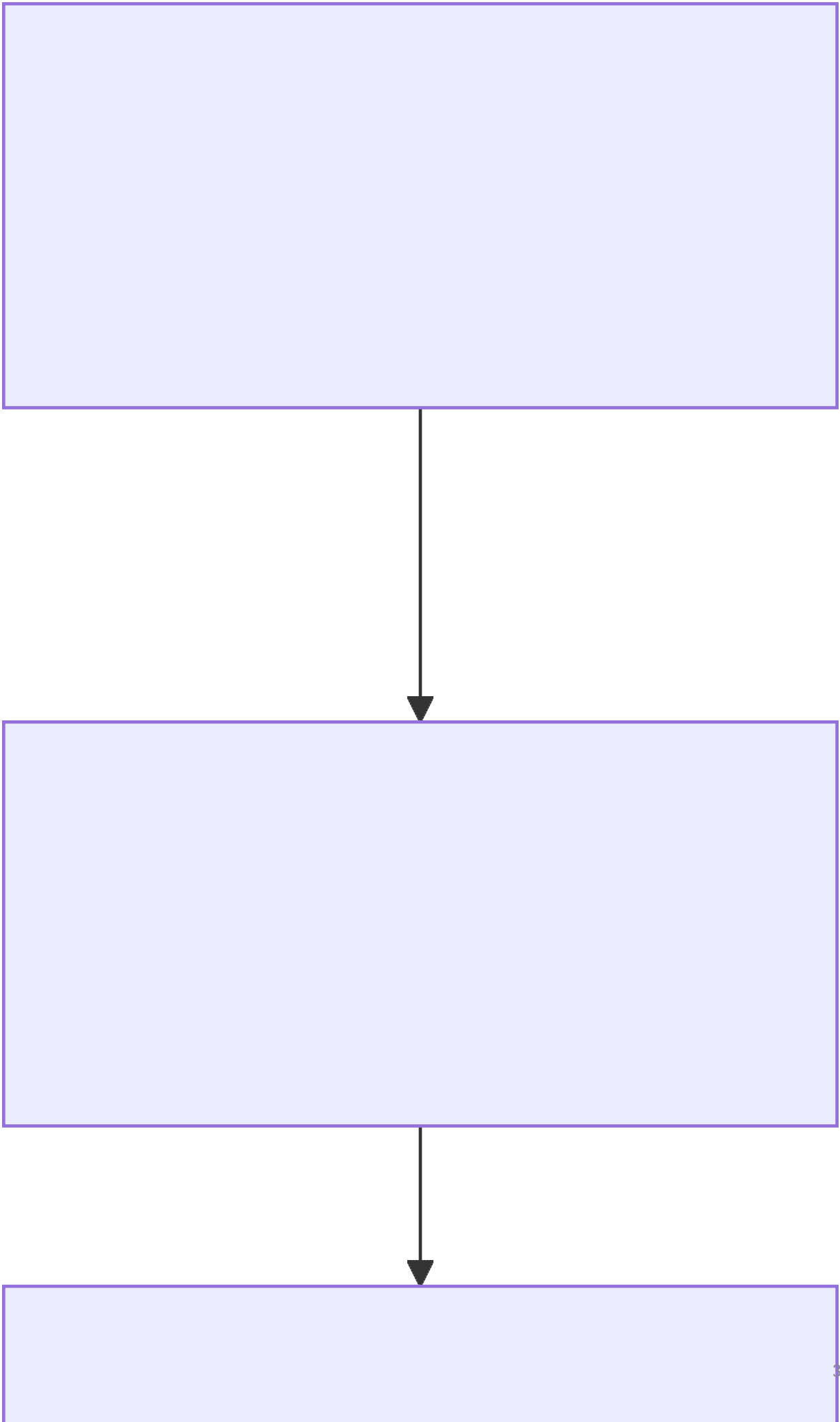


Figure 1 — Top-level platform zones: users → internal ALBs → EKS workloads → external providers

2. AWS Infrastructure Layer

2.1 Terraform Layering Strategy

Infrastructure is provisioned in three sequential Terraform layers, each storing state independently in S3 (`llmops-tfstate-492`). Separating layers means a change to the application layer does not risk destroying the VPC or EKS cluster.

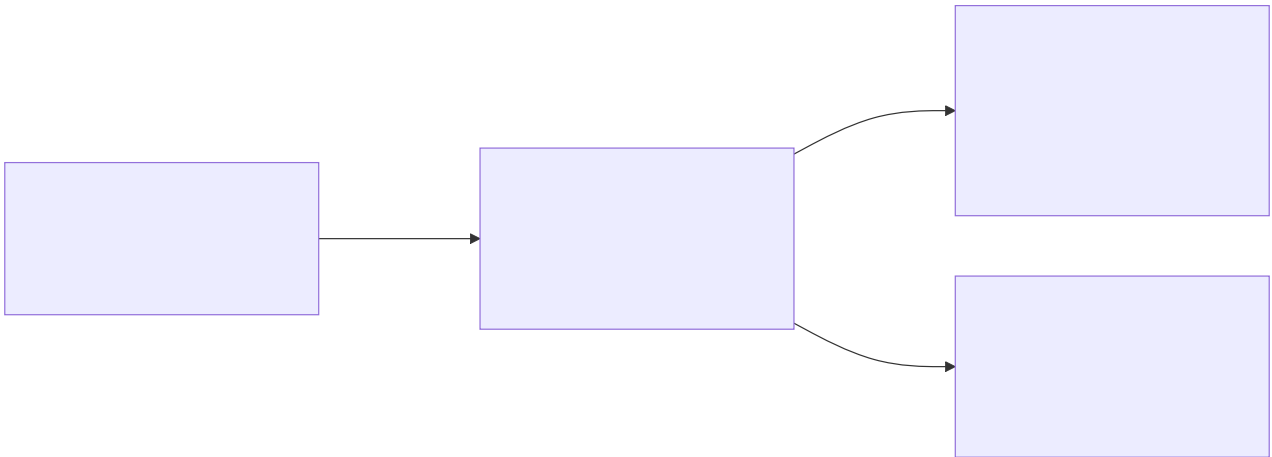


Figure 2 — Three-layer Terraform state separation

2.2 Network Topology

EKS worker nodes sit entirely in private subnets. They reach the internet (image pulls, provider API calls) via NAT Gateway. No node has a public IP, and all load balancers use `scheme: internal`.

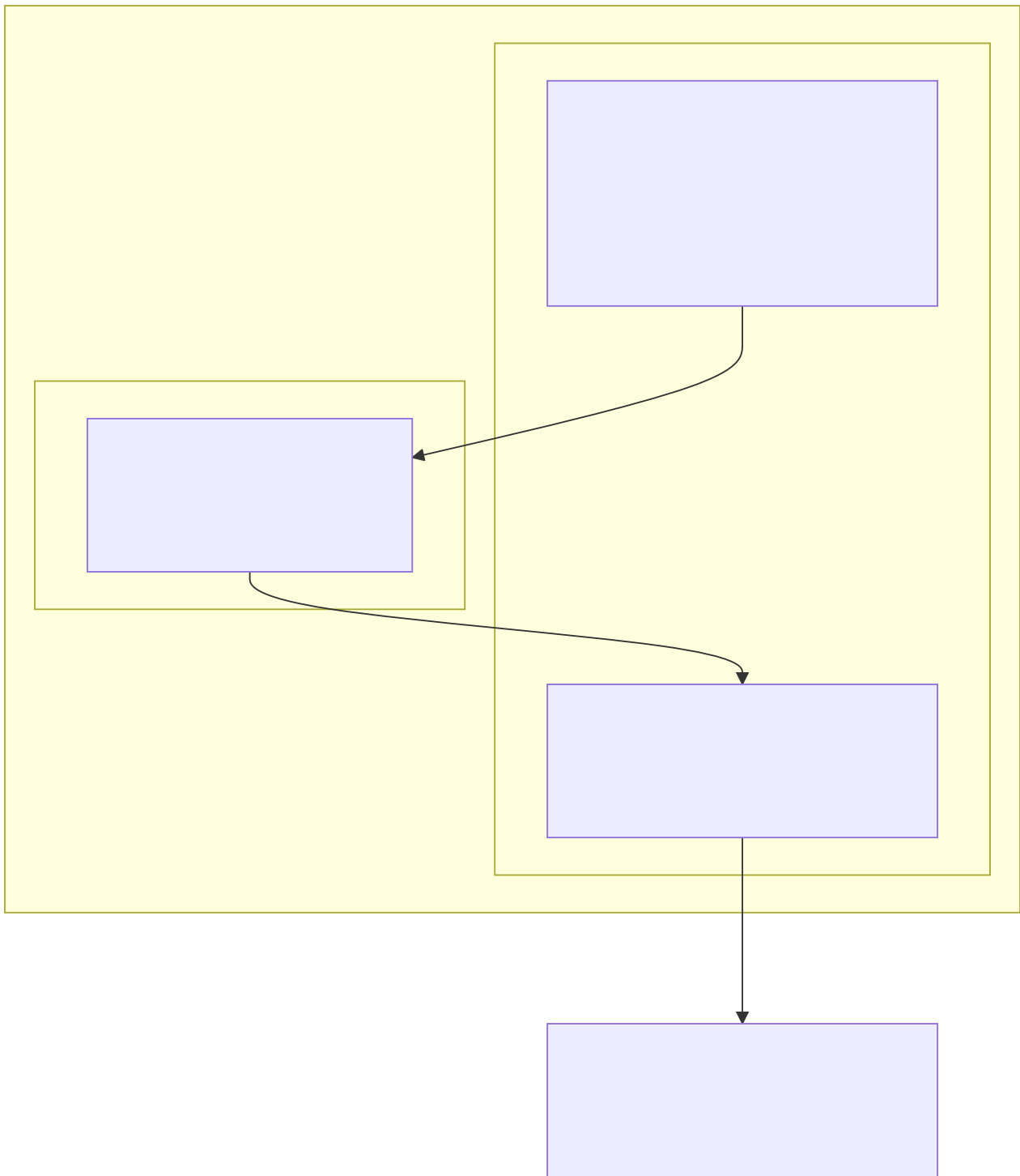


Figure 3 — VPC network topology: public subnets hold NAT + ALBs; private subnets hold EKS nodes

2.3 EKS Bootstrap Add-ons

Add-on	Purpose
AWS Load Balancer Controller	Provisions internal ALBs from Kubernetes Ingress resources
External Secrets Operator	Syncs AWS Secrets Manager values into Kubernetes Secrets
ArgoCD	GitOps controller — reconciles cluster state from Git
EBS CSI Driver	Provides <code>gp3</code> persistent volumes for PostgreSQL, ClickHouse
Cert Manager	TLS certificate management
Metrics Server	Enables HPA (CPU/memory autoscaling)

2.4 IRSA (IAM Roles for Service Accounts)

Pods access AWS services without static credentials by assuming IAM roles bound to their Kubernetes service accounts via OIDC federation. Langfuse uses IRSA to write trace blobs to S3. External Secrets Operator uses IRSA to read from Secrets Manager.

3. GitOps & Deployment Layer

3.1 ArgoCD App-of-Apps Pattern

A single root ArgoCD Application (`argocd/root-app.yaml`) watches `argocd/apps/` in Git. Each file in that directory is itself an ArgoCD Application. This gives one entry point that manages the entire platform.

Sync policy: `automated.prune=true` + `selfHeal=true` . Unauthorised changes are reverted within seconds; resources removed from Git are deleted from the cluster.

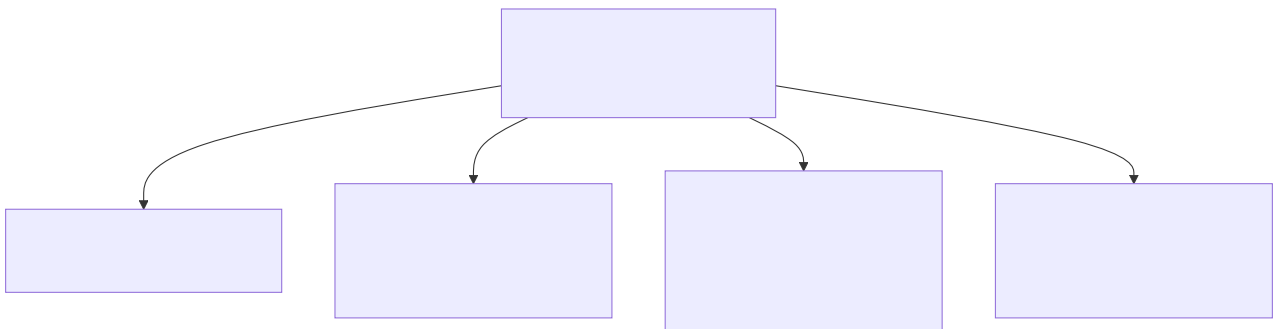


Figure 4 — ArgoCD App-of-Apps: root application manages all child apps from a single Git source

3.2 Namespace Design

Namespace	Contents
<code>litellm</code>	LiteLLM proxy (3–4 replicas)
<code>langfuse</code>	Langfuse web + worker + ClickHouse
<code>postgresql</code>	PostgreSQL (primary + metrics sidecar)
<code>redis</code>	Redis (master + metrics sidecar)
<code>ollama</code>	Ollama local model server
<code>open-webui</code>	Open WebUI (2–5 replicas, HPA)
<code>monitoring</code>	Prometheus, Grafana, Alertmanager
<code>loki</code>	Loki + Promtail
<code>external-secrets</code>	External Secrets Operator
<code>argocd</code>	ArgoCD controllers

3.3 Network Policy Map

NetworkPolicies restrict which namespaces can initiate connections. Only LiteLLM can reach PostgreSQL and Redis directly; Open WebUI can only reach LiteLLM.

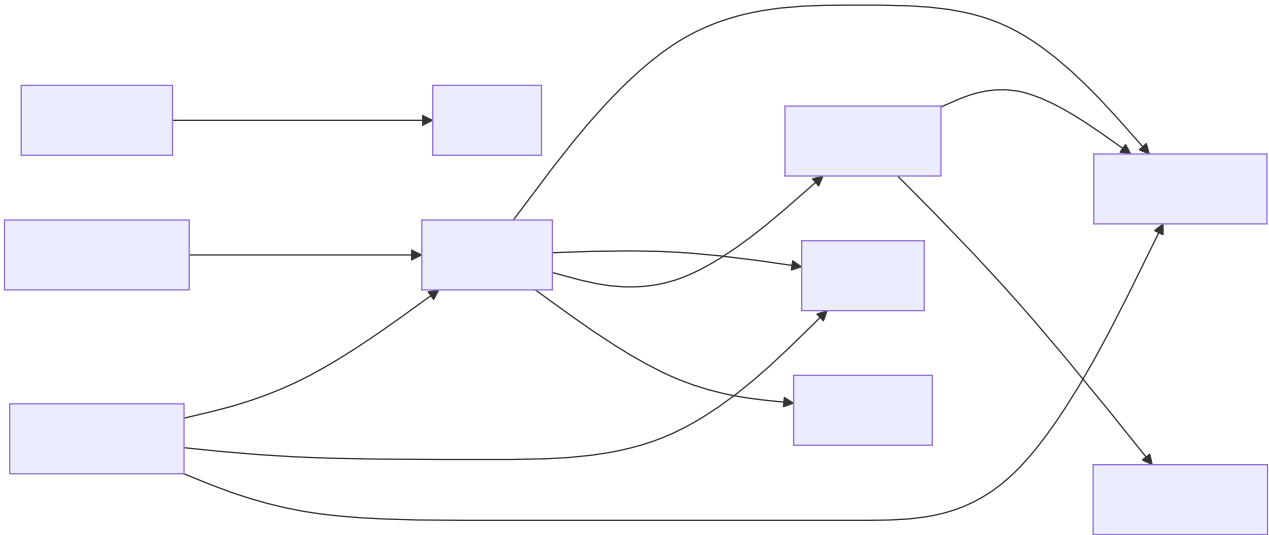


Figure 5 — Allowed namespace-to-namespace connections (all other paths are denied by default-deny policies)

3.4 Bootstrap Order



Figure 6 — Five-step cluster bootstrap sequence

4. LLM Gateway — LiteLLM

LiteLLM is the single choke point for all LLM traffic. No service — including Open WebUI — calls an external provider API directly.

4.1 Model Aliases

Alias	Backing Models	Use Case	Team Access
fast-chat	GPT-4o-mini, Gemini 2.0 Flash, Llama 3.2	General internal chat	All teams
coding-assistant	Claude Sonnet 4.6, GPT-4o-mini	Code review, debugging	Engineering only
private-chat	Ollama/Llama 3.2 (in-cluster)	Sensitive internal prompts	Engineering only
long-context	Gemini 2.0 Flash (1M ctx), GPT-4o (128k)	Large document analysis	Engineering, Executives
fallback-model	GPT-4o-mini	Last-resort backup	Internal

4.2 Request Flow

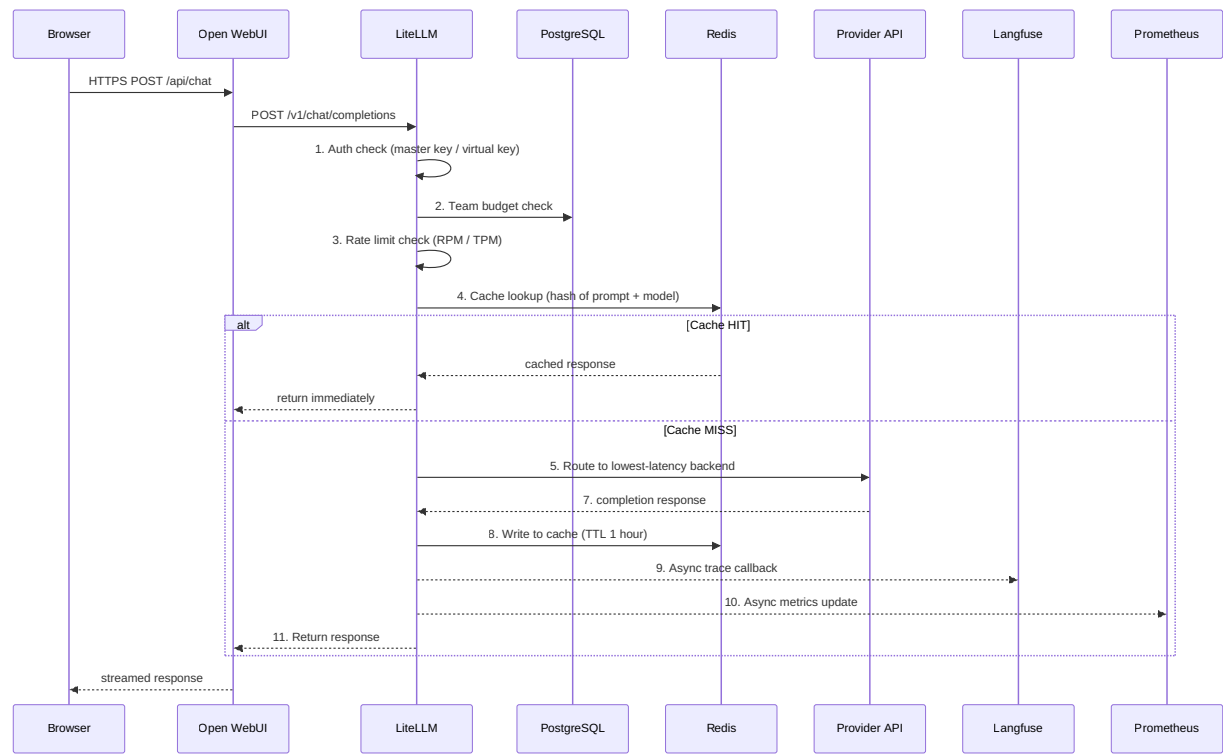


Figure 7 — End-to-end request flow: browser → Open WebUI → LiteLLM → cache check → provider → async callbacks

4.3 Fallback Chain

If a provider returns errors or times out, LiteLLM retries (2 attempts) then falls back in order. Failing providers enter a 60-second cooldown after 3 consecutive failures.

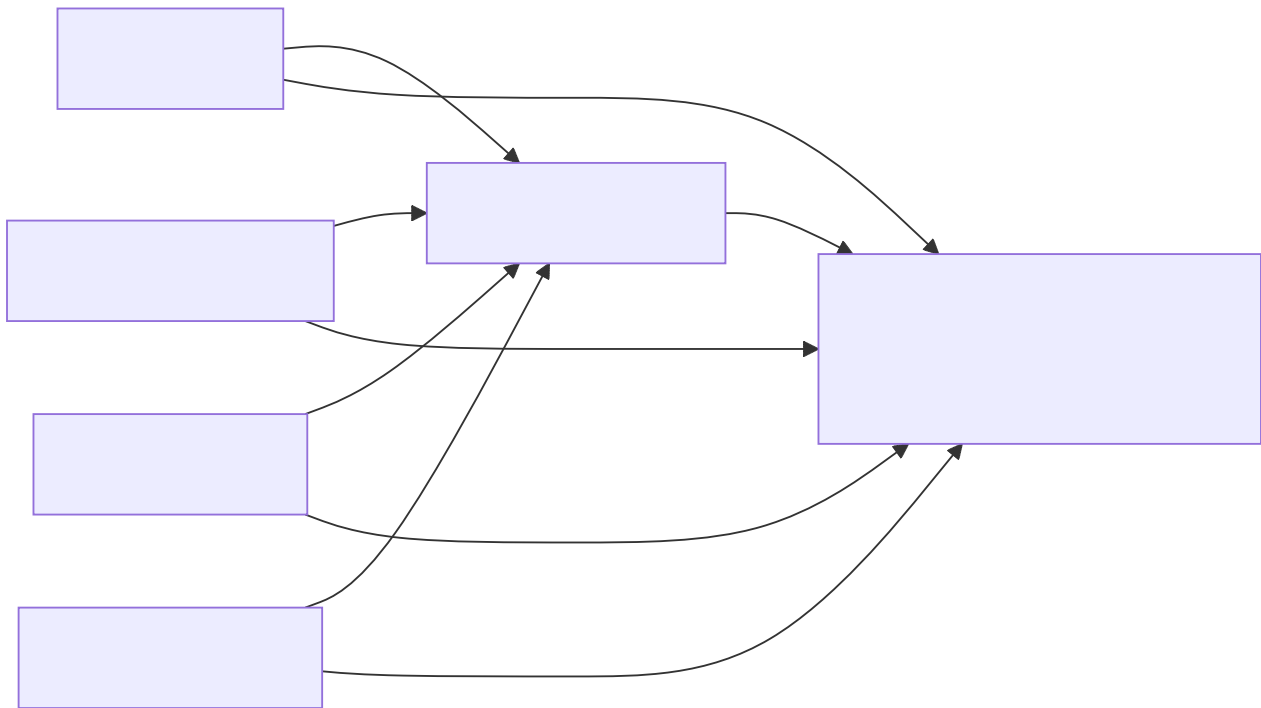


Figure 8 — Provider fallback chain: cloud aliases fall back to claude-sonnet, then to in-cluster Ollama as last resort

4.4 Team Budget & Rate Limits

Team	Monthly Budget	RPM	TPM	Allowed Models
engineering	\$100	500	500,000	All aliases
support	\$40	150	150,000	fast-chat
product	\$30	100	100,000	fast-chat
operations	\$20	80	80,000	fast-chat
executives	\$10	50	50,000	fast-chat, long-context

Platform-wide limit: 1,000 RPM / 1,000,000 TPM / \$6,000/month.

4.5 Caching

Redis stores LiteLLM response cache with a 1-hour TTL. Cache key is derived from the prompt content + model alias. The 2% duplicate-prompt rate in the traffic simulator validates this path. If Redis is unavailable, requests continue without caching.

5. Application Services

5.1 Open WebUI

ChatGPT-style web interface. Configured to speak the OpenAI-compatible API to LiteLLM — it has no direct knowledge of any model provider. Runs 2 replicas minimum, scales to 5 via HPA (CPU 70% / Memory 80%). Built-in Ollama and pipelines are disabled; LiteLLM handles all routing.

5.2 Langfuse

LLM observability platform. Receives async callbacks from LiteLLM after every request.

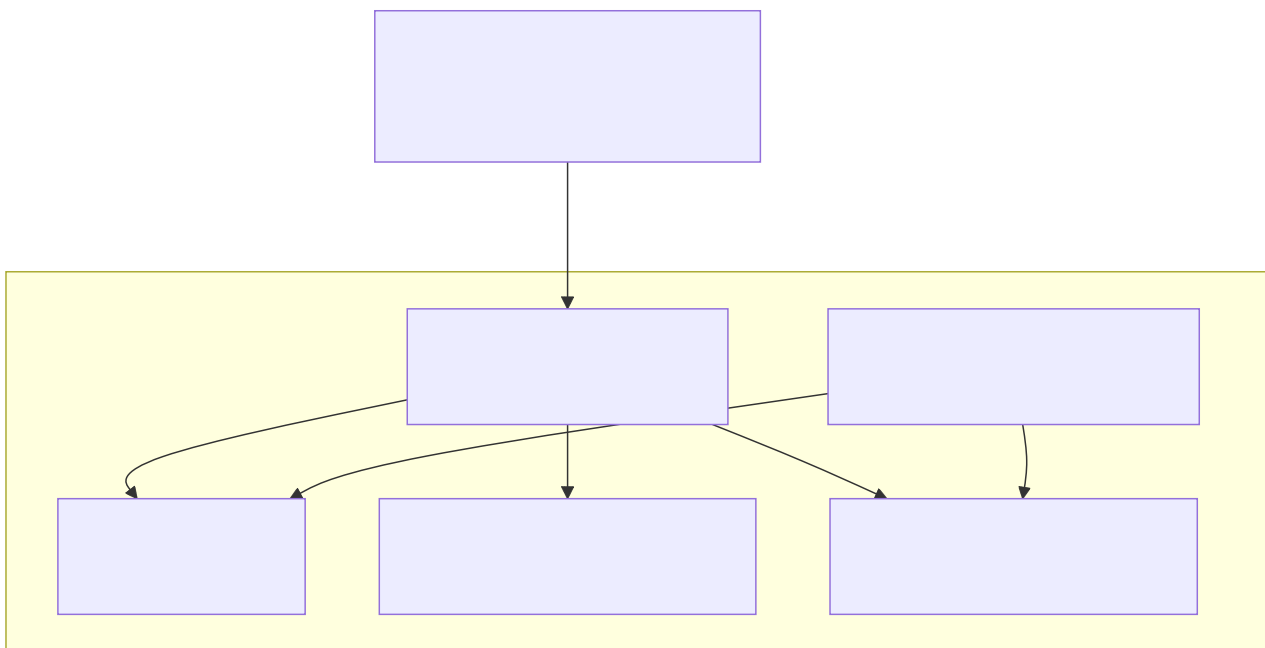


Figure 9 — Langfuse data flow: worker ingests callbacks and writes to PostgreSQL, S3, and ClickHouse

ClickHouse runs as a single node with `CLICKHOUSE_CLUSTER_ENABLED=false`. This disables `ON CLUSTER DDL` so migrations use `MergeTree` instead of `ReplicatedMergeTree`, which would hang on a single-keeper setup.

Trace fields captured: `user_id`, `team`, `model`, `prompt_tokens`, `completion_tokens`, `total_cost`, `latency_ms`, `status`, `error_message`, `trace_id`.

5.3 Ollama (Private Chat — Optional Challenge)

In-cluster model serving for the `private-chat` alias. No prompt data ever leaves the VPC for private-chat requests. Used as final fallback for all aliases when cloud providers fail. RPM limited to 10 (CPU-constrained), timeout 300s.

5.4 Redis

LiteLLM response cache. Cache key: hash(prompt + model alias). TTL: 3600 seconds. LiteLLM continues gracefully if Redis is unreachable.

5.5 PostgreSQL

Shared database for LiteLLM spend tracking and Langfuse metadata. Password injected via External Secret; `DATABASE_URL` constructed inline at runtime to avoid drift on rotation.

6. Observability Stack

Three pillars cover different failure modes — none of the three alone gives the complete picture.

Pillar	Tool	What it catches that others miss
LLM Traces	Langfuse	Prompt content, per-request cost, per-user attribution, model quality
Infrastructure Metrics	Prometheus + Grafana	CPU/memory pressure, pod restarts, queue depth, provider error rates
Application Logs	Loki + Promtail	Error stack traces, provider timeout messages, request IDs for correlation

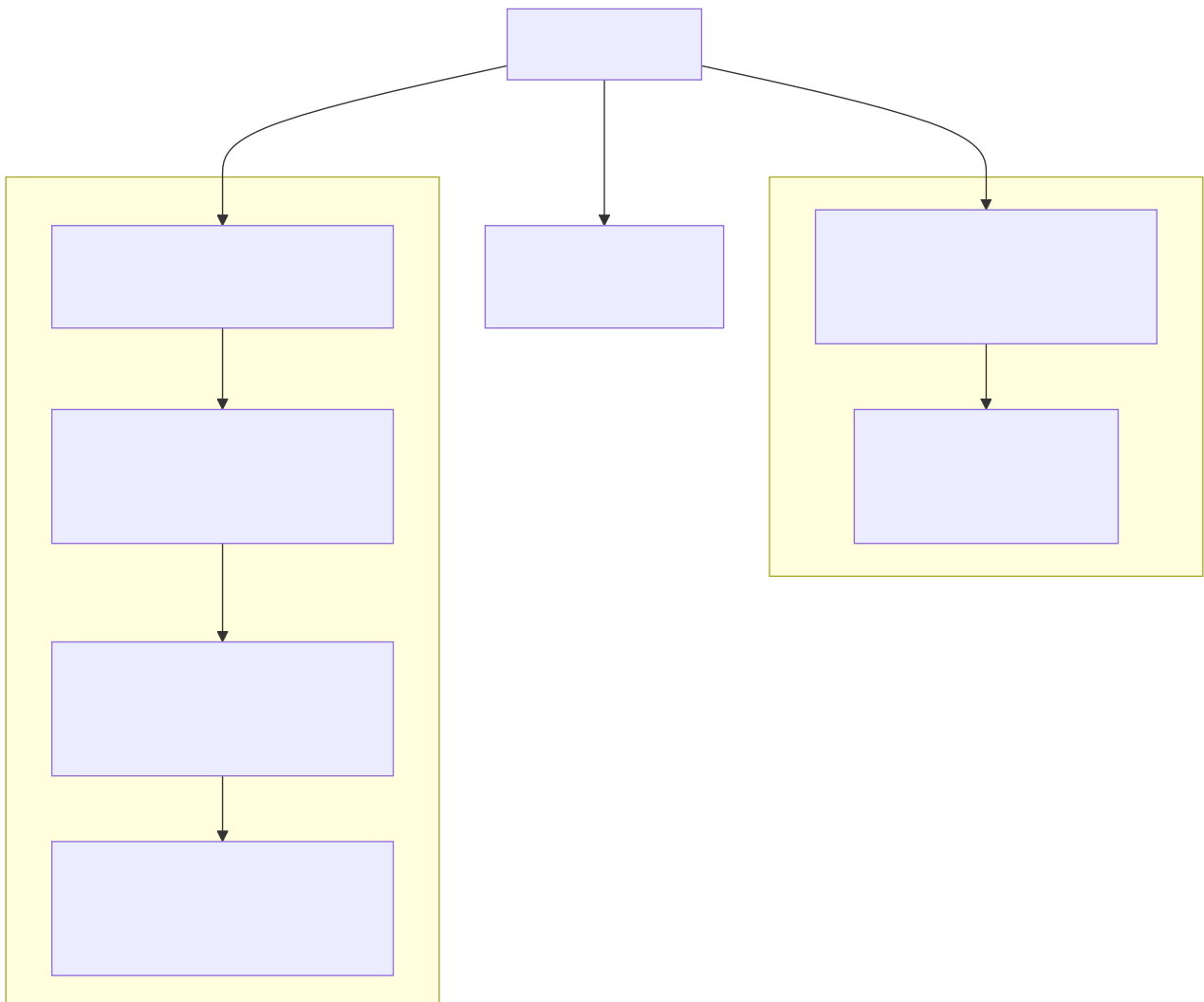


Figure 10 — Three-pillar observability: metrics pipeline (left), Langfuse traces (centre), log pipeline (right)

6.1 Alerting Rules

Alert	Condition	Severity
LiteLLMHighErrorRate	Error rate > 5% for 5m	critical
LLMProviderHighTimeoutRate	Timeout rate > 10% for 5m	warning
OpenWebUIDown	up == 0 for 2m	critical
LangfuseDown	up == 0 for 5m	warning
LiteLLMHighLatency	P95 > 3s for 10m	warning
RedisHighMemoryUsage	Memory > 80% for 5m	warning
PostgreSQLHighConnections	Connections > 80% for 5m	warning
PodCrashLooping	Restart count increase	warning

6.2 SLO Targets

Service	SLO
Open WebUI availability	99.5%
LiteLLM gateway availability	99.9%
P95 LiteLLM latency (excl. provider)	< 3 seconds
LLM request success rate	> 98%
Trace ingestion delay	< 60 seconds
Error alert detection	< 5 minutes

7. Security & Governance

7.1 Secret Lifecycle

No secrets are committed to Git. Terraform creates empty secret shells; engineers populate values manually once. External Secrets Operator re-syncs into Kubernetes Secrets on a 1-hour refresh cycle.

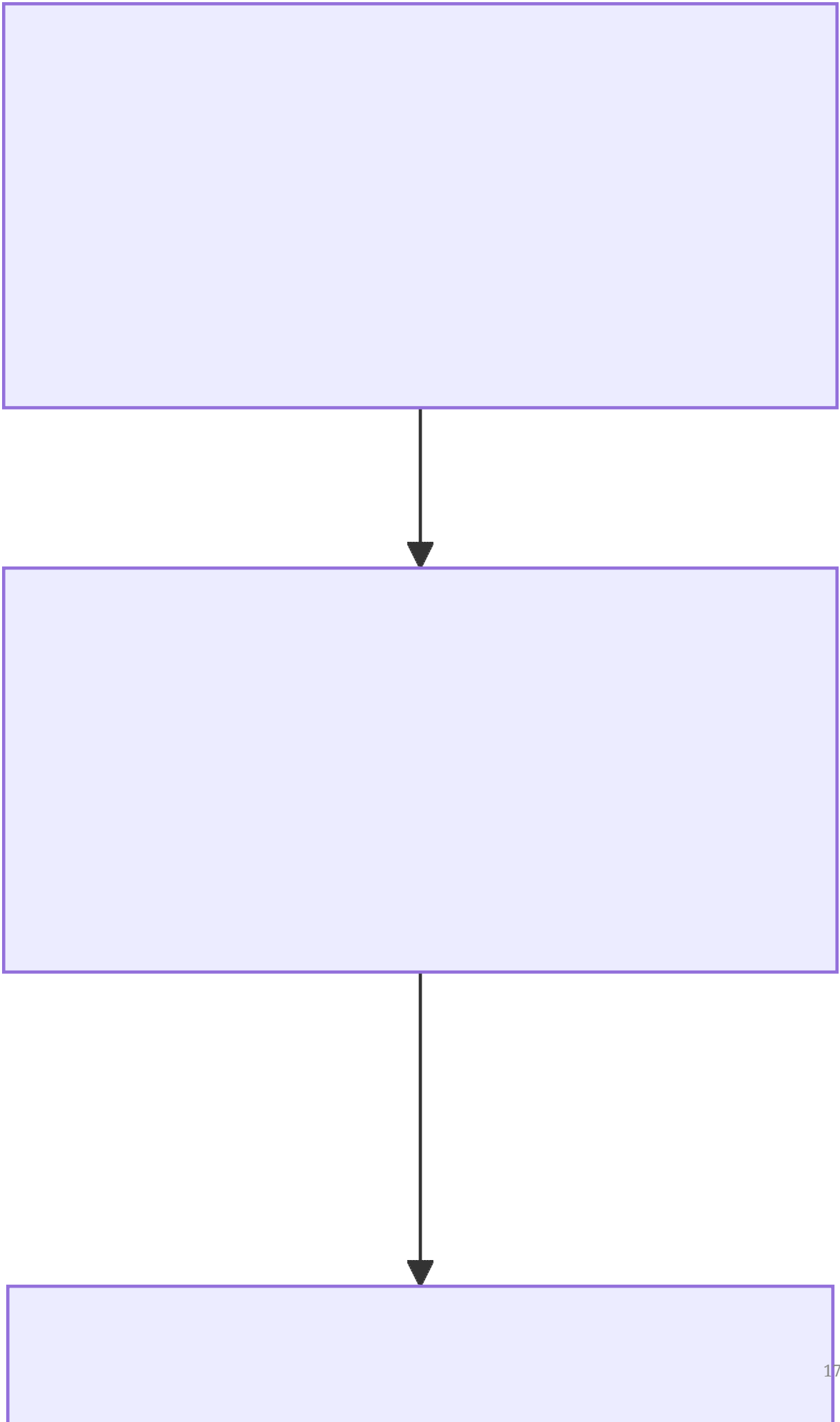


Figure 11 — Secret lifecycle: Secrets Manager → External Secrets Operator (IRSA) → Kubernetes Secret → pod env vars

7.2 Role-Based Model Access

Enforced at the LiteLLM layer via team virtual keys. Each team is created with an explicit `models` allowlist. A request for a model not in the allowlist returns HTTP 403.

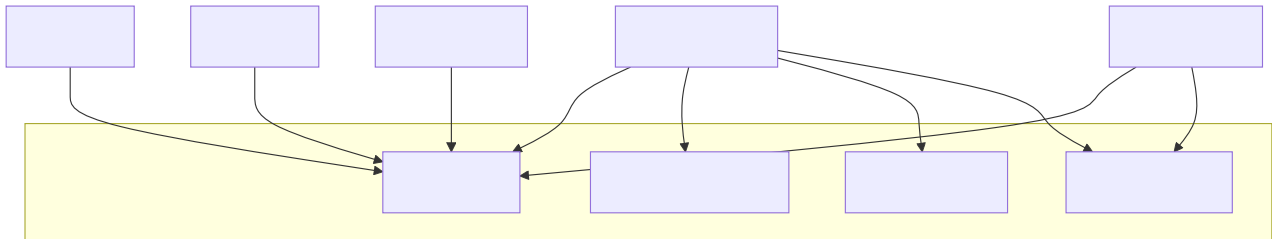


Figure 12 — Team RBAC: model alias access per team

7.3 Data Protection

- **Prompt content** stored only in Langfuse (admin-only ALB). Not written to Loki or Prometheus.
- **Exception redaction:** `redact_messages_in_exceptions: true` prevents prompt leakage in error tracebacks.
- **PII masking:** Regex guardrails configured (pre-call); currently disabled pending LiteLLM version support. Mitigation: Langfuse access restricted to platform admins.
- **Trace retention:** 30 days (Langfuse). Log retention: 14 days (Loki).
- **Separation:** Operational logs (Loki) contain only request metadata. Prompt content is isolated in Langfuse.

7.4 Network Security

- All namespaces have NetworkPolicies with a default-deny ingress rule.
- No pod has a public IP.
- LiteLLM is the only service authorised to call external provider APIs.

8. Traffic Simulation & Chaos Testing

8.1 Traffic Simulator

`argocd/traffic-simulator/simulator.py` models 290 users across 5 teams. Base load: 100 req/min. Burst windows: 10:00–10:30 SGT and 15:00–15:20 SGT at 1,000 req/min.

Traffic Type	Rate	Purpose
Long-context prompts	5%	Test <code>long-context</code> alias with large payloads
Duplicate prompts	2%	Validate Redis cache hit rate
Provider failure simulation	3%	Trigger fallback chain
Expensive-model requests	15%	Stress-test budget enforcement
Sensitive prompts (fake PII/secrets)	1%	Validate masking and detection alerts

8.2 Chaos Scenarios

Failure	Expected Platform Behaviour
Primary provider timeout	LiteLLM retries (×2) then falls back to next alias backend
Redis unavailable	Chat continues; cache misses increase latency; Redis alert fires
Langfuse unavailable	Chat continues; traces lost during outage; Langfuse alert fires
One Open WebUI pod crashes	Remaining pod serves traffic; availability stays above 99.5%
One LiteLLM pod crashes	Remaining pods serve traffic; PDB ensures min 1 available
PostgreSQL connection pressure	PG connection alert fires before connections reach max

9. Architecture Tradeoffs & Decisions

9.1 Why LiteLLM

LiteLLM provides model aliasing, fallback chains, budget enforcement, rate limiting, Redis caching, and dual Langfuse/Prometheus callbacks out of the box. Building equivalent functionality from scratch would take weeks. The tradeoff is tight coupling to LiteLLM's release cadence — upgrading requires re-validating config (the regex guardrail feature is deferred exactly because of this).

9.2 Why Open WebUI

Open WebUI is a production-grade chat interface that speaks the OpenAI-compatible API natively — session management, streaming, multi-model switching, and authentication included. Its configurable `OPENAI_BASE_API_URL` means it points at LiteLLM with one environment variable change, keeping the gateway as the sole path to providers.

9.3 Why Langfuse

Prometheus tracks *how many* requests succeeded or failed. Langfuse tracks *what was in them*: prompt content, response quality, per-user cost attribution, token breakdowns. These are different data types — Prometheus cannot store prompt text; Langfuse cannot replace time-series alerting. Both are needed.

9.4 Why ArgoCD App-of-Apps

Push-based CI creates invisible drift — if someone manually patches a pod, CI won't notice until the next deploy. ArgoCD's self-healing loop reverts unauthorised changes within seconds. A single `kubectl apply -f root-app.yaml` bootstraps the entire platform from a fresh cluster.

9.5 Why Ollama In-Cluster

The optional challenge was to add a local/private model backend. Ollama was chosen over vLLM because it runs on CPU (no GPU node required for a dev cluster) and supports Llama 3.2 with minimal configuration. The in-cluster placement means `private-chat` requests never leave the VPC. As a bonus, Ollama serves as the final fallback for all aliases during cloud provider outages.

9.6 Known Limitations & Production Recommendations

Limitation	Production Recommendation
Regex guardrails disabled (LiteLLM version gap)	Pin to a LiteLLM release that supports <code>guardrail: regex</code> ; re-enable the commented config
Single-AZ ClickHouse	Migrate to ClickHouse Cloud or a multi-replica deployment for trace durability
OIDC/SSO for Open WebUI not implemented	Integrate with corporate IdP (Okta, Google Workspace) using Open WebUI's OIDC support
Ollama on CPU only	Add GPU node group for faster private-chat inference at scale
Secret rotation is manual	Automate with AWS Lambda + Secrets Manager rotation schedule
No distributed tracing	Add OpenTelemetry collector to correlate traces across LiteLLM, Langfuse, and application layers
Static team setup via Job	Move team provisioning to Terraform or a GitOps-managed operator for full declarative control